

2. 문항 1 정답 및 해설

문제 요약: 74164 8비트 시프트 레지스터와 기본 논리 게이트를 사용하여, 현재 입력을 포함한 최근 9개 입력이 모두 1일 때 $Z=1$ 이 되는 시스템을 설계한다.

정답 논리식

$$QA = X(t-1), QB = X(t-2), \dots, QH = X(t-8)$$

$$Z = X(t) * QA * QB * QC * QD * QE * QF * QG * QH$$

해설: 74164는 직렬 입력을 받아 8개의 병렬 출력에 최근 과거 입력을 순서대로 저장하는 SIPO(Serial-In Parallel-Out) 시프트 레지스터로 사용할 수 있다. 현재 클록 주기에서 입력되는 값은 아직 8비트 레지스터 내부의 과거 출력 전체에 포함되지 않았다고 보고, 현재 입력 $X(t)$ 를 조합논리에 직접 넣는다. 따라서 현재 입력 1개와 레지스터에 저장된 과거 8개 입력이 모두 1일 때만 출력 Z 가 1이 된다.

실제 74164 계열 IC는 serial input이 A, B 형태로 제공되는 경우가 많으므로 한 입력은 X 에, 다른 입력은 논리 1에 연결하거나 두 입력을 모두 X 에 묶어 serial data가 X 가 되도록 둘 수 있다. Clear 단자는 비활성 상태로 두며, 초기 상태를 확정하려면 시작 전에 clear로 레지스터를 0으로 초기화한다.

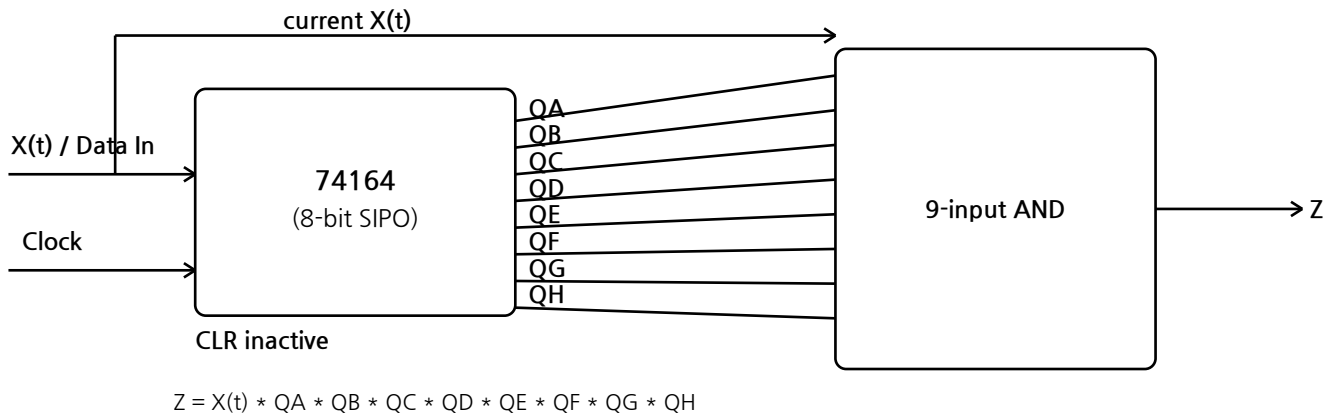


그림 1. 현재 입력 X 와 74164의 8개 병렬 출력을 모두 AND하여 최근 9개가 모두 1인지 판별한다.

주의: Z 의 타이밍은 Mealy형 조합 출력으로 해석한다. 즉, 클록 에지 직전 또는 해당 클록 주기 중의 현재 입력 $X(t)$ 와, 직전 클록까지 저장된 $QA \sim QH$ 를 함께 판별한다.

3. 문항 2-a 정답 및 해설: 4비트 동기식 카운터 사용

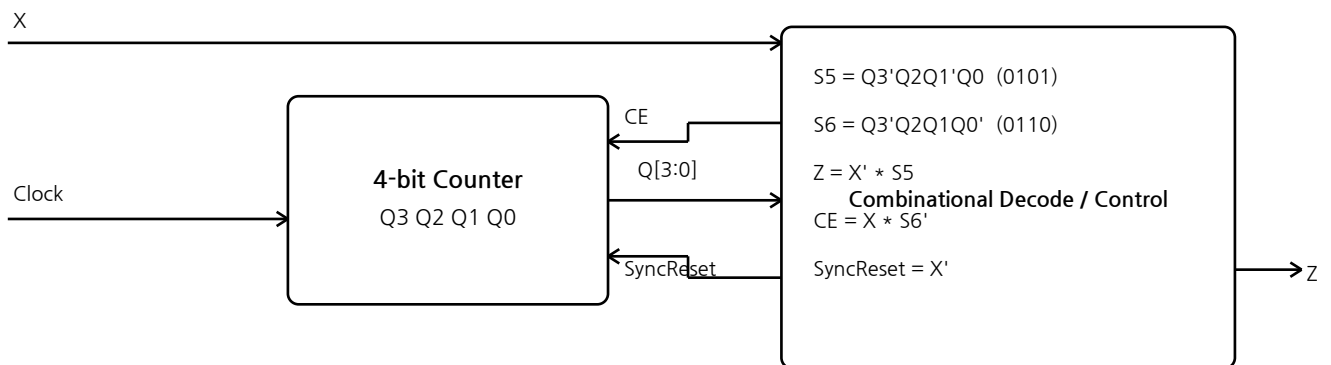
문제 요약: 입력 X가 1로 정확히 5번 연속 들어온 직후 0이 들어오는 클록 주기에서만 Z=1이 되도록 설계한다. 주 소자는 Count Enable과 동기식 Reset 단자를 가진 4비트 카운터이다.

핵심 아이디어

카운터 상태 Q3Q2Q1Q0를 “현재까지 연속으로 들어온 1의 개수”로 사용한다. X=1이면 카운트하고, X=0이면 다음 클록에서 0으로 리셋한다. 다만 “정확히 5번” 조건을 만족하려면 6개 이상 연속 1이 들어온 경우를 5로 착각하면 안 된다. 따라서 카운터가 6(0110)에 도달하면 CE를 0으로 만들어 6에서 유지시키는 포화 처리를 한다.

상태 검출식 및 출력식

$S5 = Q3' * Q2 * Q1' * Q0$ # counter state 5 = 0101
 $S6 = Q3' * Q2 * Q1 * Q0'$ # counter state 6 = 0110
 $Z = X' * S5$
 $CE = X * S6'$
 $SyncReset = X'$



State sequence for consecutive 1s: 0 → 1 → 2 → 3 → 4 → 5 → 6 → hold at 6

When the next input is 0, only state 5 makes Z=1. State 6 means “too many 1s”.

그림 2. 4비트 카운터 방식의 보정 블록도. S6 상태에서 CE를 0으로 만들어 6개 이상 연속 1을 모두 같은 “초과 상태”로 유지한다.

동작 설명: 연속된 1이 5번 들어오면 카운터는 0101 상태가 된다. 그 다음 클록 주기에 X=0이 입력되면 조합논리는 X'=1과 S5=1을 동시에 보므로 Z=1을 출력한다. 동시에 SyncReset=X'=1이므로 해당 클록 에지에서 카운터는 0으로 초기화된다. 반대로 1이 6번 이상 연속되면 카운터는 0110에서 멈추므로, 이후 0이 들어와도 S5=0이어서 Z는 1이 되지 않는다.

직전까지 연속된 1의 개수	카운터 상태	마지막 입력 X=0일 때 Z	판정
4개	0100	0	부족하므로 검출 안 함
5개	0101	1	정확히 5개이므로 검출
6개	0110	0	초과이므로 검출 안 함
7개 이상	0110에서 hold	0	초과 상태 유지

왜 단순 CE=X 방식은 위험한가? 4비트 카운터는 16을 넘으면 다시 0으로 돌아가는 modulo-16 카운터다. CE=X만 사용하면 21개의 1 뒤에 0이 들어오는 경우에도 상태가 다시 5가 되어 오검출할 수 있다. 따라서 “정확히 5번” 문제에서는 반드시 6 이상 초과 상태를 분리해야 한다.

4. 문항 2-b 정답 및 해설: 8비트 시프트 레지스터 사용

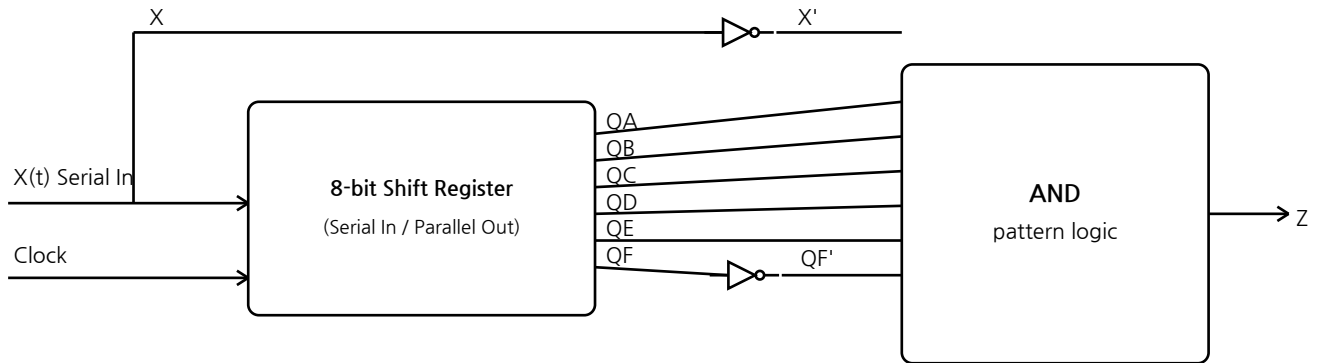
문제 요약: 8비트 직렬 입력, 병렬 출력 시프트 레지스터를 사용하여 “정확히 5번의 1 후 0” 패턴을 검출한다.

시프트 레지스터 출력은 과거 입력을 직접 보관하므로, 마지막 0이 현재 입력 $X(t)$ 로 들어오는 순간에 직전 다섯 비트와 그 이전 비트를 동시에 검사하면 된다.

출력식

$$QA = X(t-1), QB = X(t-2), QC = X(t-3), QD = X(t-4), QE = X(t-5), QF = X(t-6)$$

$$Z = X' * QA * QB * QC * QD * QE * QF'$$



$$Z = X' * QA * QB * QC * QD * QE * QF'$$

At the final 0: QA-QE are 1 and QF is 0.

그림 3. 현재 입력 X 는 0이어야 하므로 X' 를 사용하고, 직전 5개 출력 QA~QE는 모두 1이어야 하며, 6번째 과거 입력 QF는 0이어야 한다.

해설: 검출하려는 실제 패턴은 0 1 1 1 1 1 0이다. 마지막 0이 현재 입력 X 라면 직전 1~5번째 과거 입력인 QA~QE가 모두 1이어야 한다. 또한 정확히 5개의 1이어야 하므로 그보다 한 칸 앞인 QF는 0이어야 한다. 만약 $QF=1$ 이면 직전 연속 1의 개수가 6개 이상이므로 “정확히 5번” 조건에 맞지 않는다.

시간 위치	t-6	t-5	t-4	t-3	t-2	t-1	t
신호	QF	QE	QD	QC	QB	QA	X
필요 값	0	1	1	1	1	1	0

QG, QH는 이 패턴 검출에는 사용하지 않는다. 초기 상태가 문제에 영향을 주지 않게 하려면 시프트 레지스터를 시작 전에 0으로 reset/clear한 뒤 동작시키는 것이 좋다.

5. 문항 3 정답 및 해설

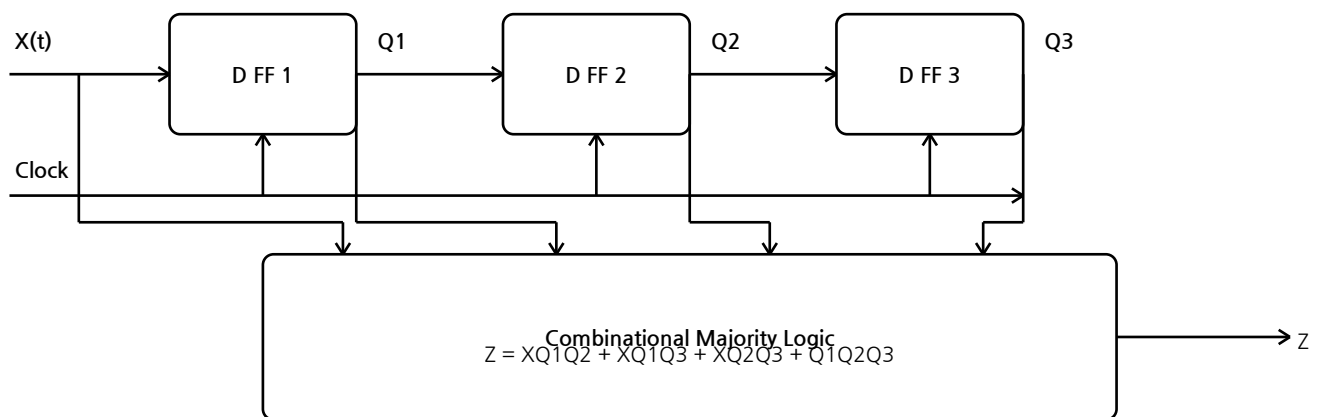
문제 요약: 현재 입력을 포함한 최근 4개의 입력 중 1이 3개 이상이면 Z=1이 되는 회로를 D 플립플롭과 조합 논리 게이트로 설계한다.

변수 정의

$Q1 = X(t-1)$
 $Q2 = X(t-2)$
 $Q3 = X(t-3)$
 최근 4개 입력 = { $X(t)$, $Q1$, $Q2$, $Q3$ }

정답 논리식

$$Z = X*Q1*Q2 + X*Q1*Q3 + X*Q2*Q3 + Q1*Q2*Q3$$



$Q1=X(t-1)$, $Q2=X(t-2)$, $Q3=X(t-3)$. All flip-flops share the same Clock.

그림 4. 세 개의 D 플립플롭으로 과거 3개 입력을 저장하고, 현재 입력 X와 함께 4입력 중 3개 이상이 1인지 조합논리로 판별한다.

해설: 최근 4개 입력 중 3개 이상이 1인 경우는 네 가지 3개 조합 중 하나 이상이 1인 경우와 같다. 즉 X,Q1,Q2가 모두 1이거나, X,Q1,Q3가 모두 1이거나, X,Q2,Q3가 모두 1이거나, Q1,Q2,Q3가 모두 1이면 된다. 네 항을 OR로 묶으면 위의 다수결 논리식이 된다. 네 비트가 모두 1인 경우에는 여러 항이 동시에 1이지만 OR 출력은 여전히 1이므로 문제없다.

1의 개수	예시	출력 Z
0, 1, 2개	0000, 1000, 1100 등	0
3개	1110, 1101, 1011, 0111	1
4개	1111	1

초기 3클럭 동안은 Q1,Q2,Q3에 충분한 과거 입력이 채워지지 않았을 수 있다. 과제에서 초기 출력까지 명확히 요구한다면 D 플립플롭들을 0으로 reset한 뒤 동작을 시작한다고 명시하면 된다.